

BookVault – Secure Privacy-First Library Management

CSE447 Lab Project | Choice 2

Stack: Express / Node.js · Supabase (PostgreSQL) · React + TypeScript · Tailwind CSS v4

All cryptographic algorithms implemented from scratch – no built-in crypto libraries.

1 Overview

BookVault is a library management web application focused on patron privacy, where all sensitive data — reading histories, checkout records, fines, book reviews, chat messages, and user profiles — is encrypted before storage and decrypted only upon authorized retrieval. Checkout records, book reviews, and fine records act as the “posts,” and patron details act as the “profile” required by the lab specification.

The goal is to build a working system where a patron’s reading habits, personal information, and communication with the librarian stay completely protected from surveillance or database leaks — even the Head Librarian (admin) cannot read patrons’ encrypted data.

2 Users & Roles

- **Regular User (Patron):** Register/login with 2FA, browse catalog, borrow and return books, pay fines, write private book reviews, chat with the librarian, manage profile, and rotate encryption keys.
- **Admin (Head Librarian):** Manage book inventory and user account status, view fine summaries, reply to patron chat messages, and view audit logs — but **cannot** decrypt patrons’ reading history, reviews, profile details, or fine record content.

3 Website Features

3.1 Patron Features

3.1.1 F1. Registration & Login

- Patron registers with: username, email, phone, library card number, and password.
- **Password:** A random 128-bit salt is generated; the password is hashed as `SHA-256(salt || password)` using a **custom SHA-256** implementation. Stored as `salt:hash`.

- **Email, phone, library card number:** Each field is individually encrypted with the patron's **RSA public key** before storage.
- Upon registration, the Key Management Module generates the patron's **RSA key pair** (for profile and chat) and **ECC key pair** (for records and reviews).
- On login: input password is hashed with the stored salt and compared. On match, a 2FA challenge is triggered.
- **Two-Factor Authentication:** A 6-digit TOTP code is generated via **HMAC-SHA256**(`shared_secret[unix_timestamp / 30]`) (from scratch). The patron must enter the code within a 60-second window. Only after **both factors pass** is a session token issued.

3.1.2 F2. Browse Book Catalog

- Patrons can search and browse the library's book collection by title, author, or genre.
- Availability status (Available / Checked Out) is displayed.
- Catalog data is **not sensitive** and is stored in plaintext — only user-generated data is encrypted.

3.1.3 F3. Borrow a Book (Checkout)

- Patron clicks "Borrow" on an available book. The system creates a checkout record with a due date (e.g., 14 days).
- The checkout record (`book_id`, `book_title`, `checkout_date`, `due_date`, `status`) is **ECC/ECIES encrypted** using the patron's ECC public key.
- An **HMAC-SHA256 tag** is computed over the ciphertext and stored alongside.
- `due_date` and `status` are also stored in plaintext so the system can enforce due dates without decryption.
- The book's available copy count is decremented.

3.1.4 F4. Return a Book

- Patron clicks "Return" on an active checkout in their dashboard.
- The system checks if the return is on time or overdue:
 - **On time:** Status updated to **returned**; encrypted record re-tagged with a new HMAC.
 - **Overdue:** Fine is calculated and a fine record is created (see Fine System).
- The book's available copy count is incremented.

3.1.5 F5. Fine System (Late Return Penalty)

When a patron returns a book after the due date, the system automatically calculates and records a fine.

Days Overdue	Fine Rate	Additional Action
1 – 7 days	5 BDT / day	—
8 – 14 days	10 BDT / day	—
15+ days	20 BDT / day	Book replacement notice issued

- Fine records are **ECC/ECIES encrypted** and **HMAC-tagged** before storage.
- Patron is **blocked from borrowing** new books until all fines are paid.
- Fine payment is simulated (patron clicks “Pay Fine” to clear the balance).
- Admin can view fine summaries (username, amount, payment status) but **cannot** decrypt fine record content.

Example: Due: Aug 1 → Returned: Aug 19 → 18 days overdue.

Fine = $(7 \times 5) + (7 \times 10) + (4 \times 20) = 35 + 70 + 80 = \mathbf{185 \text{ BDT}}$ + replacement notice.

3.1.6 F6. My Vault – Reading History & Active Borrows

- Private patron dashboard showing: currently borrowed books (with due date count-down), overdue books (highlighted with fine amount), past checkouts, outstanding fines, and private reviews.
- For each record: system fetches ciphertext → recomputes HMAC → compares → if mismatch, flags as tampered and blocks decryption → if valid, decrypts with patron’s ECC private key.
- Data is decrypted only in memory; never re-stored as plaintext.

3.1.7 F7. Write & View Private Book Reviews

- After returning a book, patrons can write private reviews or reading notes.
- Reviews are encrypted with **ECC/ECIES** and tagged with **HMAC-SHA256**.
- Only the patron can view their own reviews; admin cannot decrypt.

3.1.8 F8. Book Request via Secure Chat

- Patron can message the Head Librarian to request new books, report damaged copies, or ask about account issues.
- **Encryption:** Each message is encrypted with the recipient’s **RSA public key** (only the recipient can read it).

- **Integrity:** An **HMAC tag** is computed by the sender over the ciphertext (proves authenticity and detects tampering).
- **Replay protection:** The timestamp is included in the HMAC computation.
- DB stores in a `chat_messages` table: `sender_id`, `recipient_id`, `encrypted_body`, `hmac_tag`, `timestamp`.
- On read: HMAC verified first → if valid, decrypted with recipient's RSA private key.

3.1.9 F9. Profile Management

- Patron can view and update personal details (name, email, phone, address, library card).
- All fields are individually encrypted with **RSA** (patron's public key).
- **CBC-MAC** tag computed over the profile blob for integrity verification.
- On view: CBC-MAC verified → RSA decrypted → displayed in memory only.
- On update: new value RSA-encrypted → CBC-MAC recomputed → stored.

3.1.10 F10. Key Rotation (Privacy Reset)

- Patron can request key rotation from their security settings.
- KMM generates new RSA + ECC key pairs.
- All existing encrypted records are: decrypted with old keys → re-encrypted with new keys → HMAC/CBC-MAC recomputed.
- Old keys are permanently deleted.

3.2 Admin (Head Librarian) Features

3.2.1 A1. Book Inventory Management

Add, edit, or remove books from the catalog (title, author, ISBN, genre, copies). Only the admin can write to the catalog; patrons have read-only access.

3.2.2 A2. User Account Management

View the list of registered patrons and manage account status (suspend/reactivate). Admin can see usernames, registration dates, and account status, but **cannot** see encrypted profile fields (email, phone, library card) or reading history.

3.2.3 A3. Fine Management Dashboard

View an overview of all outstanding fines: patron username, fine amount, days overdue, and payment status. Fine record content remains encrypted.

3.2.4 A4. Secure Chat – Reply to Patrons

Read and reply to patron messages using the same RSA encryption + HMAC integrity pipeline. Admin’s replies are encrypted with the patron’s RSA public key.

3.2.5 A5. Audit Log Viewer

View a log of security-relevant events: logins, failed attempts, 2FA results, key rotations, fine payments, account changes. Logs contain event metadata only — no encrypted content.

4 The 9 Security Features

#	Security Feature	Implementation in BookVault
1	Registration & Login	Custom SHA-256 + salt for passwords; RSA encryption for user info fields.
2	Two-Factor Authentication	HMAC-based TOTP as second factor after password verification.
3	Encrypted User Profile	All profile fields individually encrypted with RSA; admin cannot read.
4	Key Management Module	Generates, stores, distributes, and rotates RSA + ECC key pairs per user.
5	Encrypted Posts (Check-out, Reviews, Fines)	All records ECC/ECIES encrypted before database write.
6	MAC Integrity Layer	HMAC-SHA256 on records, reviews, fines, and chat; CBC-MAC on profile blobs.
7	Role-Based Access Control	Patron vs. Admin privileges strictly separated and enforced server-side.
8	Secure Patron–Librarian Chat	Messages RSA-encrypted + HMAC-tagged; timestamp-based replay protection.
9	Secure Session Management	Cryptographically random tokens, HttpOnly cookies, IP fingerprinting, 30-min idle timeout, single-session enforcement.

5 Security Requirements Mapping

The following table maps each lab requirement to the specific BookVault feature(s) that satisfy it.

#	Lab Requirement	Feature(s)
1	Login & Registration modules for secure user authentication	F1 – Registration & Login
2	All user info (username, email, contact) encrypted before storage	F1 – RSA encryption on profile fields
3	Passwords hashed + salted before storage	F1 – Custom SHA-256 + 128-bit salt
4	Two-step authentication (primary credentials + second factor)	F1 – Password (factor 1) + HMAC-TOTP (factor 2)
5	Key Management Module (generation, distribution, storage, rotation)	F10 – KMM with full key life-cycle
6	Users can create, view, edit posts with all data auto-encrypted	F3 (borrow), F7 (reviews), F5 (fines), F6 (view)
7	All critical data stored encrypted; no plaintext in DB	All features – no sensitive data in plaintext
8	MAC (CBC-MAC or HMAC) for data integrity & tamper detection	F6 (HMAC on records), F9 (CBC-MAC on profile), F8 (HMAC on chat)
9	Exclusively asymmetric encryption (RSA and ECC; no symmetric)	RSA (profile, chat) + ECC/ECIES (records, reviews, fines)
10	At least two different asymmetric algorithms	RSA for profile & chat; ECC for records & reviews
11	RBAC – separate admin vs. regular user privileges	F2, A1–A5 – enforced server-side
12	Secure session management – tokens, hijack prevention	Session module – random tokens, HttpOnly, IP check, timeout

6 Cryptography Plan

All algorithms are implemented **from scratch** in pure JavaScript (Node.js). No built-in cryptographic modules or third-party encryption libraries (e.g., Node.js `crypto` module,

`bcrypt`, `jose`, `crypto-js`) are used. The only OS-level call permitted is `crypto.getRandomValues()` for entropy seeding.

- **RSA (2048-bit):** Prime generation via Miller-Rabin primality test; key generation with $e = 65537$ and d via extended Euclidean algorithm; encryption $m^e \bmod n$; decryption $c^d \bmod n$; manual OAEP-style padding.
- **ECC / ECIES (secp256k1):** Elliptic curve point addition and scalar multiplication from scratch; ECIES: generate ephemeral key pair $\rightarrow$ compute shared secret via ECDH $\rightarrow$ derive key via manual KDF $\rightarrow$ encrypt message.
- **SHA-256:** Full compression function with all 64 rounds, message scheduling, and Merkle-Damgård construction.
- **HMAC-SHA256:** $HMAC = SHA256((K \oplus opad) \| SHA256((K \oplus ipad) \| message))$.
- **CBC-MAC:** Manual block chaining with 128-bit blocks; XOR each block with previous ciphertext; final block serves as the MAC tag.
- **TOTP:** $OTP = HMAC - SHA256(secret, \lfloor unix_time/30 \rfloor) \bmod 10^6 \rightarrow$ 6-digit code.

7 Data Flow Summary

Patron Action	Crypto Applied	DB Stored As
Register (profile)	RSA encrypt each field	Ciphertext + CBC-MAC tag
Set password	SHA-256(salt password)	salt:hash
Borrow book	ECC/ECIES encrypt record	Ciphertext + HMAC tag
Return overdue	ECC/ECIES encrypt fine	Ciphertext + HMAC tag
Write review	ECC/ECIES encrypt text	Ciphertext + HMAC tag
Send chat message	RSA encrypt body	Ciphertext + HMAC tag
Login attempt	SHA-256 verify	Compare only
OTP entry	HMAC-SHA256 verify	Compare only
View My Vault	HMAC verify $\rightarrow$ ECC decrypt	In-memory only
View profile	CBC-MAC verify $\rightarrow$ RSA decrypt	In-memory only

8 Tech Stack

Layer	Technology
Backend	Express.js on Node.js
Database	Supabase (PostgreSQL) – stores only ciphertext + MAC tags + metadata
Frontend	React with TypeScript + Tailwind CSS v4
Cryptography	100% from scratch in JavaScript (no <code>crypto</code> module)
Session Store	Supabase (PostgreSQL <code>sessions</code> table)
Deployment	Vercel

Important note: Supabase is used **solely as a PostgreSQL database** via its client SDK for data storage and retrieval. Supabase’s built-in authentication (`supabase.auth`), Row Level Security policies, and Edge Functions are **not used** — all authentication, encryption, and access control logic is implemented from scratch in the Express backend.

9 RBAC Privilege Matrix

Action	Patron	Admin
Register / Login		
View own profile (decrypted)		×
Borrow / return books		×
Add / edit checkout records & reviews		×
Browse book catalog		
Add / edit / delete books	×	
Manage user account status	×	
View encrypted record metadata	×	
Send / receive secure chat		
Pay fines		×
View fine management dashboard	×	
Rotate own keys		×
View audit logs	×	